

Software Requirements Specification

A Mobile Application for Facility Management

Team

.

Members

Joy Bassem | 13003606 (Team
Lead)

Hana Khaled | 13002847

Malak Fawzy | 13007189

Maryam Basim | 13005731

Mohamed Adel | 13002519

Adam Sherbiny | 13002518

Tutorial Details

T4 | Hassan Abdelqader

1. Introduction

Campus environments require continuous monitoring and maintenance to ensure that facilities remain safe, clean, and functional for students and staff. However, issues such as broken furniture, flickering lights, plumbing problems, or cleanliness concerns often go unreported or take a long time to be addressed due to the lack of a structured communication channel between the university community and the facility management team.

The CampusCare system is proposed as a digital solution to address this problem. It is a mobile application designed to bridge the communication gap between members of the university community and the Facility Management team by allowing issues to be reported, tracked, and resolved efficiently.

The purpose of this system is to provide an easy and efficient platform through which users can report infrastructure or maintenance problems on campus. Through this platform, the Facility Management team can receive reports in real time, assign workers to resolve issues, and update the status of maintenance tasks. This improves response time and helps maintain a clean, safe, and well-functioning campus environment.

The intended audience for this document includes software developers, project stakeholders, system administrators, and members of the university management responsible for implementing and maintaining the system. It also serves as a guide for testers and future developers who may expand the system.

The scope of the CampusCare system includes the development of a mobile application and a backend system that supports reporting, tracking, and managing campus maintenance issues. The system will allow community members to report problems by submitting descriptions, photos, and location information, while facility management staff can monitor issues, assign tasks to workers, and update the status of repairs.

1.1 Product Vision & Scope

The main goal of the CampusCare system is to provide a comprehensive digital platform that improves the way campus maintenance issues are reported, managed, and resolved. The system enables members of the university community to easily report infrastructure or cleanliness issues while allowing the Facility Management team to efficiently manage and track these reports.

One of the key objectives of the system is to improve communication between community members and the facility management team by providing a structured and formal reporting channel. Instead of issues remaining unnoticed or being communicated informally, users can report them directly through the application.

Another goal of the system is to reduce response time for maintenance issues. By allowing real-time submission and tracking of problems, the facility management team can quickly review incoming reports, assign tasks to workers, and monitor the progress of repairs.

The system also aims to improve campus maintenance efficiency by creating a centralized database of reported issues. This data can help identify recurring problems, support preventive maintenance, and assist management in making informed decisions regarding campus infrastructure.

Additionally, the system promotes community participation and shared responsibility. By empowering students and staff to report issues they encounter, the system encourages active involvement in maintaining the campus environment.

1.2 Definitions and Acronyms

SRS: Software Requirements Specification

CampusCare: The mobile system used to report and manage campus maintenance issues

Ticket: A digital record representing a maintenance issue reported through the system

Community Member: A student or staff member who can submit maintenance tickets

Facility Manager: A user responsible for reviewing tickets and assigning workers

Worker: A user responsible for resolving assigned maintenance issues

Admin: A system administrator who manages users, roles, and locations

RBAC: Role-Based Access Control, a security method that restricts system access based on user roles

API: Application Programming Interface used for communication between system components

React Native: Framework used to build the mobile application

Node.js: Technology used to develop the backend server

PostgreSQL: Database system used to store application data

2. User Stories

-GIU Community Member User Story:

US-CM-01:

As a Community Member, I should be able to create an account so that I can log in to the application.

US-CM-02:

As a Community Member, I should be able to log in using my email and password so that I can access the application

US-CM-03:

As a Community Member, I should be able to submit an issue with its details (location, photo, and category) so that the problem can be reported accurately.

US-CM-04:

As a Community Member, I should be able to add a description to the issue so that the worker understands the problem clearly.

US-CM-05:

As a Community Member, I should be able to view my submitted issues so that I can track their status.

US-CM-06:

As a Community Member, I should be able to log out so that my data remains secure on shared devices.

US-CM-07:

As a Community Member, I should be able to view the status of my issue (Pending, In Progress, Resolved) so that I know the progress of my request.

US-CM-08:

As a Community Member, I should receive a notification when the status of my issue changes so that I stay informed about the progress.

-Facility Manager User Story:

US-FM-01:

As a Facility Manager, I should be able to create an account so that I can log in to the application.

US-FM-02:

As a Facility Manager, I should be able to log in using my email and password so that I can access the application.

US-FM-03:

As a Facility Manager, I should be able to log out so that my data remains secure on shared devices.

US-FM-04:

As a Facility Manager, I should be able to view all submitted issues so that I can assign them to workers.

US-FM-05:

As a Facility Manager, I should be able to update an issue's status so that the system reflects the current state of the issue.

US-FM-06:

As a Facility Manager, I should be able to assign an issue to a worker so that the worker can start fixing it.

US-FM-07:

As a Facility Manager, I should be able to close an issue after it is resolved so that the system reflects that the issue is completed.

US-FM-08:

As a Facility Manager, I should receive a notification when a worker marks a task as completed so that I can verify the work.

US-FM-09:

As a Facility Manager, I should be able to set a priority level for an issue so that urgent problems are addressed first.

US-FM-10:

As a Facility Manager, I should be able to filter issues by status, category, location, or priority so that I can manage maintenance tasks efficiently.

-Worker User Story:

US-W-01:

As a Worker, I should be able to create an account so that I can log in to the application.

US-W-02:

As a Worker, I should be able to log in using my email and password so that I can access the application.

US-W-03:

As a Worker, I should be able to log out so that my data remains secure on shared devices.

US-W-04:

As a Worker, I should be able to view my assigned issues so that I can start working on them.

US-W-05:

As a Worker, I should be able to comment on an issue to indicate that the work is completed so that the facility manager can update its status.

US-W-06:

As a Worker, I should be able to mark an issue as "In Progress" so that the facility manager and community member know that it is being worked on.

US-W-07:

As a Worker, I should be able to upload a photo after fixing the issue so that the facility manager can verify the work.

US-W-08:

As a Worker, I should receive a notification when a ticket is assigned to me so that I can begin working on it.

-Admin User Story:

US-SA-01:

As a System Admin, I should be able to log in so that I can access the application.

US-SA-02:

As a System Admin, I should be able to log out so that my data remains secure on shared devices.

US-SA-03:

As a System Admin, I should be able to activate or deactivate any account so that I can ensure all users follow the system rules.

US-SA-04:

As a System Admin, I should be able to view all registered users so that I can manage the system users.

2.1 Functional Requirements

-Community Member Functional Requirements:

FR-CM-01: Account Creation

The system shall allow a Community Member to create an account by providing required information (e.g., name, email, and password).

FR-CM-02: User Authentication

The system shall allow a Community Member to log in using a valid email and password.

FR-CM-03: Issue Submission

The system shall allow a Community Member to submit an issue including the following details:

- Location
- Photo
- Category
- Description

FR-CM-04: Issue Description

The system shall allow a Community Member to add a textual description when submitting an issue.

FR-CM-05: View Submitted Issues

The system shall allow a Community Member to view a list of issues they have submitted.

FR-CM-06: Logout

The system shall allow a Community Member to log out of the application.

FR-CM-07: View Issue Status

The system shall display the current status of an issue to the Community Member.

Possible statuses include:

- Pending
- In Progress
- Resolved

FR-CM-08: Status Notification

The system shall notify a Community Member when the status of their submitted issue changes.

-Facility Manager Functional Requirements:**FR-FM-01: Account Creation**

The system shall allow a Facility Manager to create an account.

FR-FM-02: User Authentication

The system shall allow a Facility Manager to log in using a valid email and password.

FR-FM-03: Logout

The system shall allow a Facility Manager to log out of the application.

FR-FM-04: View All Issues

The system shall allow a Facility Manager to view all submitted issues in the system.

FR-FM-05: Update Issue Status

The system shall allow a Facility Manager to update the status of an issue.

FR-FM-06: Assign Issue to Worker

The system shall allow a Facility Manager to assign an issue to a specific worker.

FR-FM-07: Close Issue

The system shall allow a Facility Manager to close an issue once it has been resolved.

FR-FM-08: Completion Notification

The system shall notify a Facility Manager when a worker marks an issue as completed.

FR-FM-09: Set Issue Priority

The system shall allow a Facility Manager to assign a priority level to an issue.

FR-FM-10: Filter Issues

The system shall allow a Facility Manager to filter issues by status, category, location, or priority.

-Worker Functional Requirements:**FR-W-01: Account Creation**

The system shall allow a Worker to create an account.

FR-W-02: User Authentication

The system shall allow a Worker to log in using a valid email and password.

FR-W-03: Logout

The system shall allow a Worker to log out of the application.

FR-W-04: View Assigned Issues

The system shall allow a Worker to view issues assigned to them.

FR-W-05: Comment on Issue

The system shall allow a Worker to add comments to an issue.

FR-W-06: Update Issue Status to In Progress

The system shall allow a Worker to mark an assigned issue as In Progress.

FR-W-07: Upload Completion Photo

The system shall allow a Worker to upload a photo after completing the work on an issue.

FR-W-08: Assignment Notification

The system shall notify a Worker when a new issue is assigned to them.

-Admin Functional Requirements**FR-SA-01: Admin Authentication**

The system shall allow a System Admin to log in.

FR-SA-02: Logout

The system shall allow a System Admin to log out.

FR-SA-03: Account Management

The system shall allow a System Admin to activate or deactivate user accounts.

FR-SA-04: View Registered Users

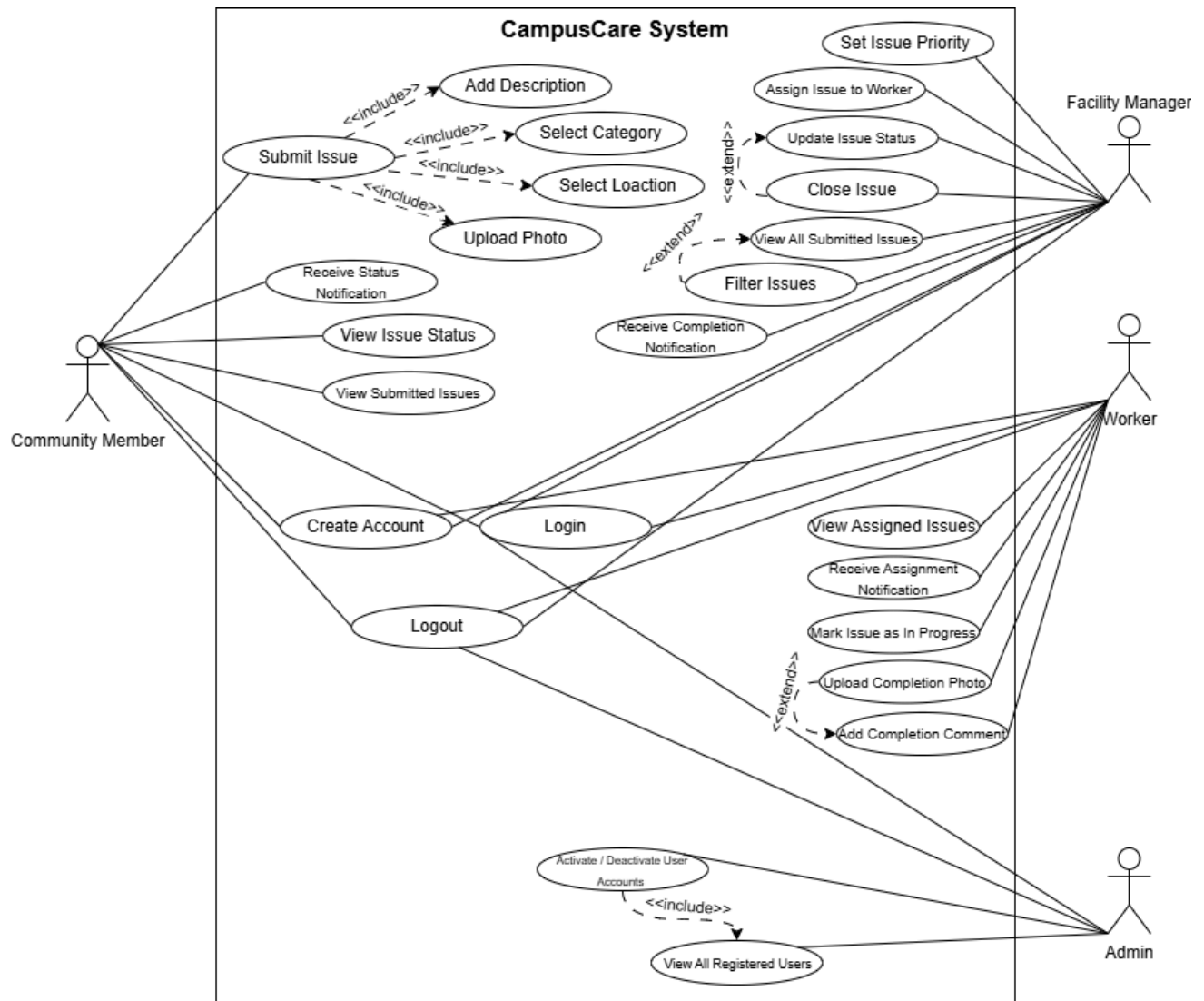
The system shall allow a System Admin to view all registered users in the system.

2.2 Product Backlog

Backlog ID	User Story ID	Priority
Core System Features		
PB-01	US-CM-01	High
PB-02	US-CM-02	High
PB-03	US-CM-03	High
PB-04	US-CM-04	High
PB-05	US-CM-05	High
PB-06	US-CM-07	High
PB-07	US-FM-01	High
PB-08	US-FM-02	High
PB-09	US-W-01	High
PB-10	US-W-02	High
Issue Management Features		
PB-11	US-FM-04	High
PB-12	US-FM-06	High
PB-13	US-W-04	High
PB-14	US-W-06	High
PB-15	US-W-05	Medium
PB-16	US-W-07	Medium
PB-17	US-FM-05	High
PB-18	US-FM-07	High
PB-19	US-FM-09	Medium
PB-20	US-FM-10	Medium
Authentication & Security		

PB-21	US-CM-06	Medium
PB-22	US-FM-03	Medium
PB-23	US-W-03	Medium
Notifications		
PB-24	US-CM-08	Medium
PB-25	US-FM-08	Medium
PB-26	US-W-08	Medium
System Administration		
PB-27	US-SA-01	Low
PB-28	US-SA-02	Low
PB-29	US-SA-03	Low
PB-30	US-SA-04	Low

2.3 UML Use-Case Diagram



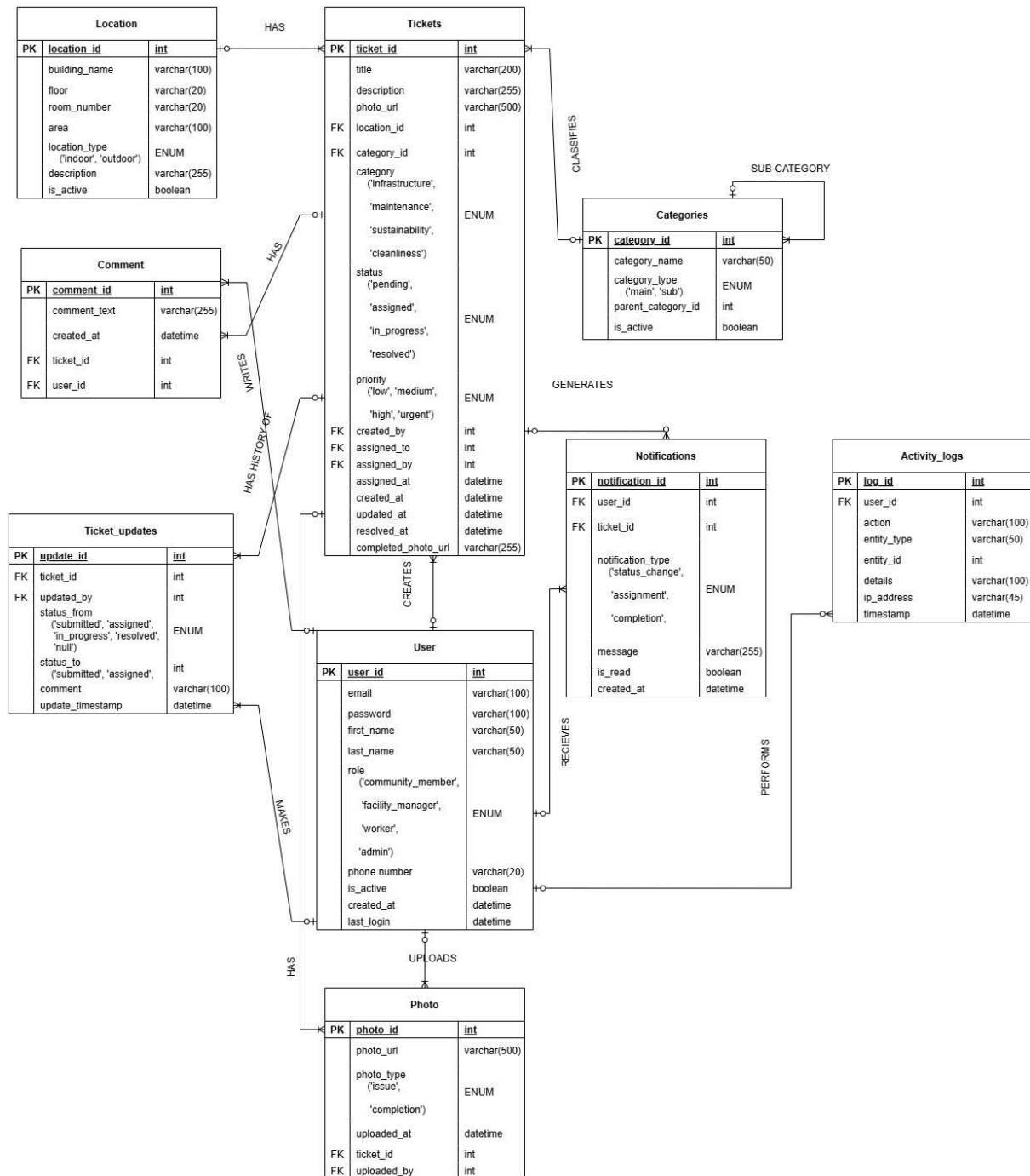
3. Non-Functional Requirements

- Response Time: Page load times under three seconds
- Concurrent Users: Support for at least 500 simultaneous users
- API Security: Rate limiting and input validation
- Backup & Recovery: Automated daily backups with disaster recovery plan
- Usability: Intuitive UI such that tickets for issues can be submitted in under one minute.
- Reliability: Must handle image uploads robustly, even on unstable network connections.

4. Constraints

The mobile application must be developed using React Native with Expo, while the backend must be implemented using Node.js with RESTful APIs and a PostgreSQL database. Image files uploaded with maintenance tickets must be stored using a cloud-based storage service. The system must support mobile devices and handle up to 500 concurrent users while maintaining acceptable performance. Security measures such as authentication and role-based access control (RBAC) must be implemented. The system also depends on stable internet connectivity for submitting tickets, uploading images, and receiving notifications.

5. Entity Relation Diagram



6. Technology Stack

- Mobile Application: React Native with Expo on top
- Backend: Node.js with RESTful APIs
- Database: PostgreSQL
- Image storage: Cloud-based storage service

7. Future Scope of the Project

- Predictive Maintenance: Analyze historical ticket data to predict recurring maintenance issues.
- Gamification: Introduce points or badges for users who report valid issues.
- Real-Time Ticket Tracking: Allow users to track repair progress in real time.
- Advanced Notifications: Add push notifications, email, or SMS alerts for ticket updates.
- Management Dashboard: Develop a web dashboard for facility managers to monitor system activity.
- AI Image Classification: Use AI to automatically detect issue categories from uploaded images.